LAB EXERCISE 2

ENCRYPTION – CRYPTO 101 IN TRYHACKME PLATFORM

AIM

To understand the fundamentals of cryptography by performing hands-on exercises in the Crypto 101 room on the TryHackMe platform, including encoding, encryption, classical ciphers, hashing, and cryptographic concepts.

TOOLS / PLATFORM REQUIRED

Web Browser (Chrome / Firefox)
Internet Connection
TryHackMe Account (Free)
TryHackMe AttackBox or Local Linux Machine

PROCEDURE

STEP 1: LOGIN TO TRYHACKME

Open a web browser

Go to:
https://tryhackme.com
Login using your registered credentials

STEP 2: OPEN CRYPTO 101 ROOM

Use the search bar and type Crypto 101
Click on the room named Crypto 101
Click Join Room

STEP 3: START ATTACKBOX

Click Start AttackBox / Start Machine
Wait until the virtual environment loads

STEP 4: PERFORM CRYPTOGRAPHY TASKS

4.1 ENCODING & DECODING:

(a) BASE64 ENCODING / DECODING

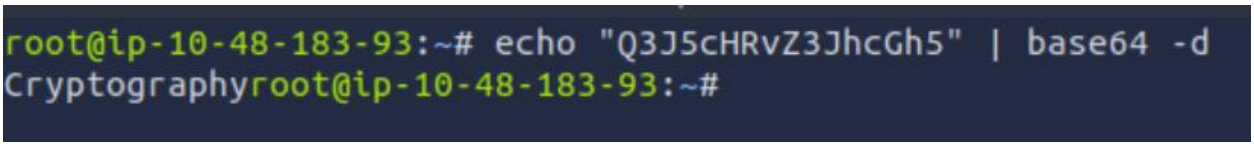
Purpose:
Base64 is an encoding technique used to convert binary data into readable ASCII format.
Decode Base64

Command:

echo "Q3J5cHRvZ3JhcGh5" | base64 -d

Output:

Cryptography
Encode Base64



Command:

echo "cryptography" | base64

Output:
Y3J5cHRvZ3JhcGh5Cg==

(b) HEXADECIMAL CONVERSION

Purpose:
Hexadecimal is a base-16 encoding system widely used in computing.

Hex to Text
echo 48656c6c6f | xxd -r -p

Output:
Hello

Text to Hex
echo "Hello" | xxd -p
Output:

```
root@ip-10-48-183-93:~# echo "Hello" | xxd -p
48656c6c6f0a
```

4.2 CLASSICAL CIPHERS
(a) CAESAR CIPHER

Purpose:
Caesar cipher shifts each letter by a fixed number.

Command (Shift = 3):

echo "KHOOR" | tr 'A-Z' 'X-ZA-W'

Output:
HELLO

```
root@ip-10-48-183-93:~# echo "KHOOR" | tr 'A-Z' 'X-ZA-W'
HELLO
```

(b) SHIFT-BASED ENCRYPTION

Purpose:
A generalized Caesar cipher using different shift values.

Method:

Try different shift values
Identify readable plaintext

Example Result:

Shift 1 → Incorrect
Shift 3 → Meaningful text → Correct
Output:

```
root@ip-10-48-183-93:~# echo "HELLO" | tr 'A-Z' 'ZABCDEFGHIJKLMNOPQRSTUVWXY'
GDKKN
```

(c) PLAYFAIR CIPHER

Purpose:
Playfair cipher encrypts pairs of letters using a key matrix.

STEP 1: CREATE FILE

nano playfair_cipher.py

STEP 2: PLAYFAIR CIPHER CODE

```
import argparse
import re

def normalize(text):
    text = re.sub(r'[^A-Za-z]', '', text).upper()
    return text.replace('J', 'I')

def build_matrix(key):
    key = normalize(key)
    alphabet = "ABCDEFGHIKLMNOPQRSTUVWXYZ"
    seen = []
    for c in key + alphabet:
        if c not in seen:
            seen.append(c)
    matrix = [seen[i:i+5] for i in range(0, 25, 5)]
    pos = {matrix[r][c]: (r, c) for r in range(5) for c in range(5)}
    return matrix, pos

def digraphs(text):
    text = normalize(text)
    i = 0
    pairs = []
    while i < len(text):
        a = text[i]
        b = text[i+1] if i+1 < len(text) else 'X'
        if a == b:
            pairs.append((a, 'X'))
            i += 1
        else:
            pairs.append((a, b))
            i += 2
    return pairs

def encrypt(text, key):
    matrix, pos = build_matrix(key)
```

```
    result = ""
    for a, b in digraphs(text):
        ra, ca = pos[a]
        rb, cb = pos[b]
        if ra == rb:
            result += matrix[ra][(ca+1)%5] + matrix[rb][(cb+1)%5]
        elif ca == cb:
            result += matrix[(ra+1)%5][ca] + matrix[(rb+1)%5][cb]
        else:
            result += matrix[ra][cb] + matrix[rb][ca]
    return result

def decrypt(text, key):
    matrix, pos = build_matrix(key)
    result = ""
    for i in range(0, len(text), 2):
        a, b = text[i], text[i+1]
        ra, ca = pos[a]
        rb, cb = pos[b]
        if ra == rb:
            result += matrix[ra][(ca-1)%5] + matrix[rb][(cb-1)%5]
        elif ca == cb:
            result += matrix[(ra-1)%5][ca] + matrix[(rb-1)%5][cb]
        else:
            result += matrix[ra][cb] + matrix[rb][ca]
    return result

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("mode", choices=["enc", "dec"])
    parser.add_argument("-k", "--key", required=True)
    parser.add_argument("-t", "--text", required=True)
    args = parser.parse_args()

    if args.mode == "enc":
        print(encrypt(args.text, args.key))
    else:
        print(decrypt(args.text, args.key))
```

STEP 3: SAVE FILE
Press CTRL + X
Press Y → Enter

STEP 4: EXECUTE PLAYFAIR CIPHER

Encryption

python3 playfair_cipher.py enc -k "MONARCHY" -t "hide the gold"

Output:

```
root@ip-10-48-183-93:~# nano playfair_cipher.py
root@ip-10-48-183-93:~#
root@ip-10-48-183-93:~#
root@ip-10-48-183-93:~# python3 playfair_cipher.py enc -k "MONARCHY" -t "hid
e the gold"
BFCKPDFIMPBZ
```

Decryption

python3 playfair_cipher.py dec -k "MONARCHY" -t
"BMNDZBXDKYBEJVDMUIXMMOUVIF"
nam

(d) VIGENÈRE CIPHER

Instruction:
Follow the same steps as Playfair and create vigenerecipher.py.
Use Python logic for shifting characters using a repeating key.
CODE:

```
import argparse

def generate_key(text, key):
    key = key.upper()
    key_new = ""
    j = 0
    for i in range(len(text)):
        if text[i].isalpha():
            key_new += key[j % len(key)]
            j += 1
        else:
            key_new += text[i]
    return key_new

def encrypt(text, key):
    result = ""
    key = generate_key(text, key)

    for i in range(len(text)):
```

```
            if text[i].isupper():
                result += chr((ord(text[i]) + ord(key[i]) - 2*ord('A')) % 26 + ord('A'))
            elif text[i].islower():
                result += chr((ord(text[i]) + ord(key[i]) - 2*ord('a')) % 26 + ord('a'))
            else:
                result += text[i]
        return result

    def decrypt(text, key):
        result = ""
        key = generate_key(text, key)

        for i in range(len(text)):
            if text[i].isupper():
                result += chr((ord(text[i]) - ord(key[i]) + 26) % 26 + ord('A'))
            elif text[i].islower():
                result += chr((ord(text[i]) - ord(key[i]) + 26) % 26 + ord('a'))
            else:
                result += text[i]
        return result

    if __name__ == "__main__":
        parser = argparse.ArgumentParser(description="Vigenere Cipher")
        parser.add_argument("mode", choices=["enc", "dec"], help="enc = encrypt,
    dec = decrypt")
        parser.add_argument("-k", "--key", required=True, help="Cipher key")
        parser.add_argument("-t", "--text", required=True, help="Text")

        args = parser.parse_args()

        if args.mode == "enc":
            print(encrypt(args.text, args.key))
        else:
            print(decrypt(args.text, args.key))
```

Command:

python3 vigenerescipher.py enc -k "KEY" -t "HELLO WORLD"

Output:

```
root@ip-10-48-183-93:~# python3 vigenerecipher.py enc -k "KEY" -t "HELLO WOR
LD"
RIJVS UYVJN
```

4.3 HASHING

(a) HASH ALGORITHM IDENTIFICATION

Hash Type	Length
MD5	32 characters
SHA-1	40 characters
SHA-256	64 characters

(b) GENERATING HASHES

```
echo -n "hello" | md5sum
echo -n "hello" | sha1sum
echo -n "hello" | sha256sum
```

4.4 ENCRYPTION CONCEPTS

(a) ENCRYPTION vs HASHING

Encryption	Hashing
Reversible	Irreversible
Uses keys	No keys
Secure communication	Password storage

(b) SYMMETRIC vs ASYMMETRIC

Symmetric	Asymmetric
One key	Two keys
Faster	Slower

STEP 5: VERIFY COMPLETION

Ensure all room tasks are marked Completed

Observe the success message in the Crypto 101 room

OUTPUT:

Successfully decoded Base64 and Hex values

Successfully executed Caesar and Playfair ciphers

Successfully generated and identified hashes

```
root@ip-10-48-183-93:~# echo -n "hello" | md5sum
5d41402abc4b2a76b9719d911017c592  -
root@ip-10-48-183-93:~# echo -n "hello" | sha1sum
aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d  -
root@ip-10-48-183-93:~# echo -n "hello" | sha256sum
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e730433629
```

RESULT:

Thus, the Crypto 101 experiment was successfully completed on the TryHackMe platform. This experiment provided hands-on understanding of encoding, encryption, classical ciphers, hashing, and cryptographic fundamentals.